

[译]使用eBPF（绕过 TCP/IP）加速云原生应用程序的经验教训

2021/11/01 (<https://www.cnxct.com/lessons-using-ebpf-accelerating-cloud-native-zh/>)

CFC4N (<https://www.cnxct.com/author/admin/>)

译者注

本文翻译自 使用eBPF加速云原生应用程序的经验教训 (<https://cyral.com/blog/lessons-using-ebpf-accelerating-cloud-native/>)，前篇文章为How to use eBPF for accelerating Cloud Native applications (<https://cyral.com/blog/how-to-ebpf-accelerating-cloud-native/>)，已经由国内博主ArthurChiao (<http://arthurchiao.art/about/>)翻译+实践整理给出：[译] 利用 ebpf sockmap/redirection 提升 socket 性能 (<http://arthurchiao.art/blog/socket-acceleration-with-ebpf-zh/>)



([//image.cnxct.com/2021/11/ebpf.png](https://image.cnxct.com/2021/11/ebpf.png))

本文对比了eBPF绕过TCP/IP栈与普通TCP协议两种模式下，网络通讯的吞吐量、延迟、事务量等指标。其技术原理在上篇中已经讲过，本篇的技术难度不大，译者参考原文，进行环境搭建，编码测试，根据自己的测试结果，对原文进行更加深刻的解读。

eBPF与主机安全

译者从事IDC的安全防御检测类产品研发，包括主机层面的入侵检测系统等。IDC领域，随着容器化技术不断演进，早已从主机时代转化到容器时代。而主机安全的研发思维，还停留在主机时代。比如：

- 物理机、虚拟机时代：进程间通讯是IPC、unixdomain socket，而容器化下，是GRPC\HTTP。
- 容器化时代：利用高效的eBPF技术做网络隔离，重新规划，能否应付得来
- 容器化时代：同一个宿主机上多个容器通讯，TCP改为本地socket转发（类似本文的技术）

当容器化技术依赖越来越多的内核新技术eBPF后，HIDS产品的检测技术，该如何演进？

木马后门利用了eBPF技术后，端口复用等问题变得相当容易，也没有独立的进程，没特征，HIDS产品该如何去发现？

翻译目的

带着这些问题，译者开始学习eBPF相关知识，为了更快更好的掌握eBPF，译者采用边学边动手的方式，来加快对新技术的理解，加深知识点的记忆。在学习过程过，感谢博主ArthurChiao (<http://arthurchiao.art/about/>) 的一系列博文。

译者的开发环境

- MacOS Big Sur 11.6
- VMware Fusion 12.1.2
- UBUNTU 21.04 server版的虚拟服务器（分配2核CPU，4G内存）

安装步骤

镜像地址：

<http://tw.archive.ubuntu.com/ubuntu-cd/hirsute/ubuntu-21.04-live-server-amd64.iso> (<http://tw.archive.ubuntu.com/ubuntu-cd/hirsute/ubuntu-21.04-live-server-amd64.iso>)

安装相应工具包、类库

```
sudo apt-get install -y make gcc libssl-dev bc libelf-dev libcap-dev clang gcc-multilib llvm libncurses5-dev git pkg-config libmnl-dev bison flex graphviz
```

```
sudo apt-get install -y make gcc clang llvm git pkg-config dpkg-dev gcc-multilib
```

修改apt源

修改源 /etc/apt/sources.list ，便于下载kernel源码

```
deb-src http://archive.ubuntu.com/ubuntu (http://archive.ubuntu.com/ubuntu) hirsute main
```

```
deb-src http://archive.ubuntu.com/ubuntu (http://archive.ubuntu.com/ubuntu) hirsute- main
```

```
sudo su
apt update
apt-get source linux-image-$(uname -r)
apt-get source linux-image-unsigned-$(uname -r)
apt install libbfd-dev libcap-dev zlib1g-dev libelf-dev libssl-dev
```

编译 bpftool 工具

```
cd /home/cfc4n/download/linux-5.11.0/tools/bpf/bpftool
make
sudo make install
```

编译bpf字节码

```
cd ~/project/os-eBPF/sockredir
cfc4n@vmubuntu:~/project/os-eBPF/sockredir$ clang -O2 -g -target bpf -I /usr/include/linux/ -I /usr/src/linux-headers-5.11.0-37/include/ -c bpf_sockops_v4.c -o bpf_sockops_v4.o
```

加载bpf字节码

```
root@vmubuntu:/home/cfc4n/project/os-eBPF/sockredir# bpftool prog load bpf_sockops_v4.o "/sys/fs/bpf/bpf_sockops"
libbpf: elf: skipping unrecognized data section(8) .rodata.str1.1

root@vmubuntu:/home/cfc4n/project/os-eBPF/sockredir# bpftool cgroup attach "/sys/fs/cgroup/unified/" sock_ops pinned
"/sys/fs/bpf/bpf_sockops"
```

测试工具

netperf是本文的测试工具，安装方式

```
sudo apt install netperf
netperf -V
Netperf version 2.7.0
```

操作系统版本

```
uname -a
Linux vmubuntu 5.11.0-37-generic #41-Ubuntu SMP Mon Sep 20 16:39:20 UTC 2021 x86_64 x86_64 x86_64 GNU/Linux
```

Linux内核TCP参数

拥塞控制

```
cat /proc/sys/net/ipv4/tcp_congestion_control
cubic
```

接收缓冲区 buffer size 最小、默认、最大

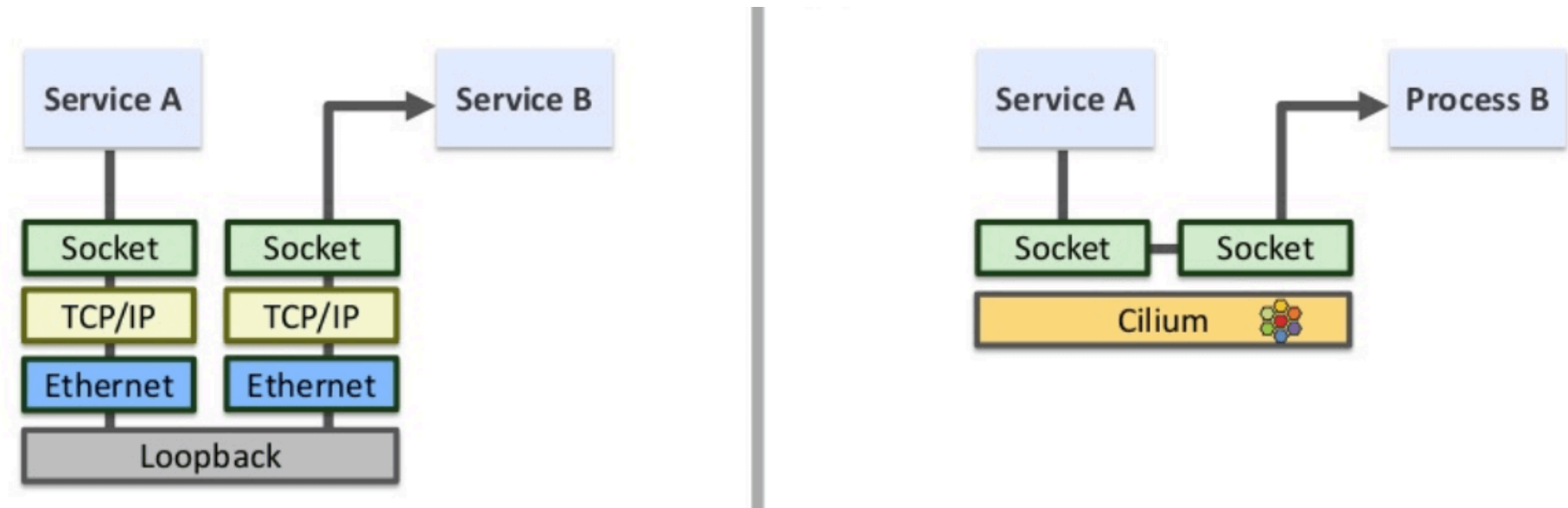
```
cfc4n@vmubuntu:~$ cat /proc/sys/net/ipv4/tcp_rmem
4096      131072  6291456
```

发送缓冲区

```
cfc4n@vmubuntu:~$ cat /proc/sys/net/ipv4/tcp_wmem
4096      16384    4194304
```

eBPF TCP链路绕过实现

译文 (<http://arthurchiao.art/blog/socket-acceleration-with-ebpf-zh/>)与原文 (<https://cyral.com/blog/how-to-ebpf-accelerating-cloud-native/>) 展示了如何编写简单的BPF程序实现 socket level 重定向 (redirection)。对于源和目的端都在同一台宿主机器上的应用来说，可以实现绕过整个TCP/IP协议栈，直接将数据发送到 socket对端。效果如下图



译者注：在大公司的基础组件部门，可以在宿主机上，自动实现网络加速，不需要业务层根据不同环境而去分别实现unix domain socket和TCP两种socket。大大减轻业务压力，提升基础架构部门的效率。

原文

我们多数客户对我们的服务有极高的性能SLA需求，我们也不断投资于性能增强以提高我们的服务质量。我们研究的其中一项技术就是eBPF，用于加速后端各种微服务之间的通信。

由于业界很少有文章全面介绍如何使用eBPF，我们决定与大家分享我们学到的知识。

这是由两部分组成的博客系列的第二部分，我们将在其中详细分享我们如何利用eBPF 进行网络加速以及我们在此过程中所做的一些有趣观察。

在之前的博文中，我们分享 如何编写用于套接字数据重定向的 eBPF代码 (<https://cyril.com/blog/how-to-ebpf-accelerating-cloud-native>) 以及如何使用 bpftool (<https://manpages.ubuntu.com/manpages/focal/man8/bpftool-prog.8.html>) 将生成的eBPF 字节码注入内核。在这篇文章中，我们深入探讨了从常规 TCP/IP 切换到eBPF后的各种性能提升。

使用 eBPF 进行网络加速——性能评估

一旦我们可以使用eBPF实现绕过TCP/IP堆栈，就必须看到性能提升后的效果。为此，我们使用了较为知名的测量网络性能的工具netperf (<https://github.com/HewlettPackard/netperf>)来评估吞吐量、延迟和事务率方面的收益。我们在以下两种情况下使用netperf评估了这些指标：

1. 流量正常通过TCP/IP堆栈，
2. 流量使用eBPF sockhash映射重定向绕过TCP/IP堆栈

测试配置

测试设置如下：

- 在 MacOS High Sierra 上运行的 VirtualBox 6.1 上的 Ubuntu Bionic 18.04.03 LTS
- Linux 内核 5.3.0-40-generic
- TCP 设置
 - 拥塞控制：cubic
 - recv() 调用的默认缓冲区大小：131072 Bytes
 - send() 调用的默认缓冲区大小：16384 Bytes
 - MSS最大报文段长度（Maximum Segment Size）：65483 Bytes

- netperf 2.6.0

测试步骤

网络吞吐测试

使用 netserver 服务端和 netperf 客户端进行60秒的各种发送消息大小的吞吐量测试：

```
netserver -p 1000
netperf -H 127.0.0.1 -p 1000 -l 60 -- -m $msg_size
```

网络延迟测试

使用 netperf 命令，执行时长参数为60秒的各种请求和响应消息大小，进行延迟测量（第 50、90 和 99%）：

```
netserver -p 1000
netperf -P 0 -t TCP_RR -H 127.0.0.1 -p 1000 -l 60 -- -r $req_size, $resp_size -o P50_LATENCY, P90_LATENCY, P99_LATENCY
```

网络事务率测试

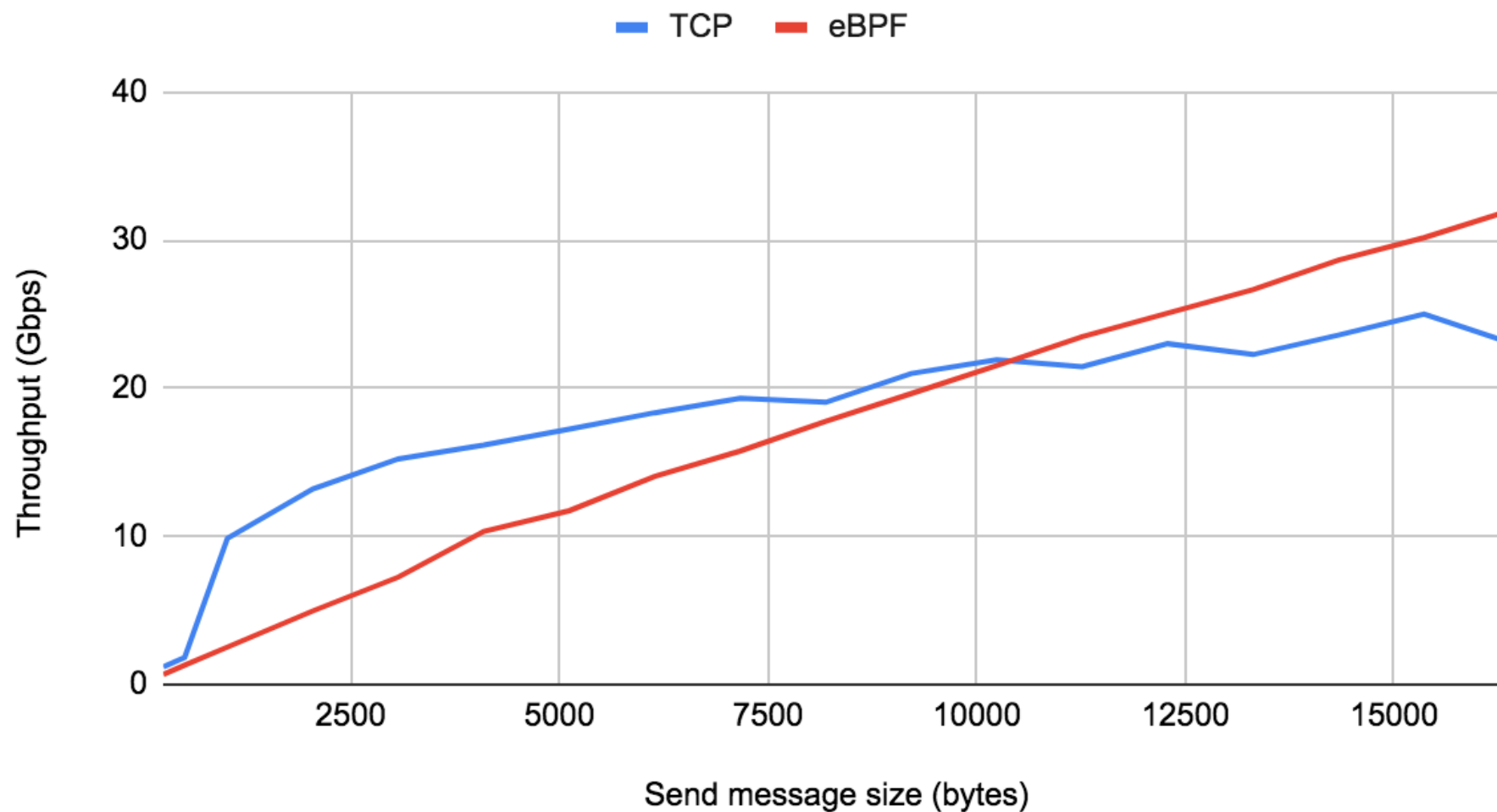
使用 netperf 命令来测试60秒运行的各种请求和响应消息大小的事务率：

```
netserver -p 1000
netperf -t TCP_RR -H 127.0.0.1 -p 1000 -l 60 -- -r $req_size, $resp_size
```

测试结果

网络吞吐量结果

TCP vs eBPF



吞吐量：eBPF sockhash修改过的TCP/IP与常规TCP/IP比较

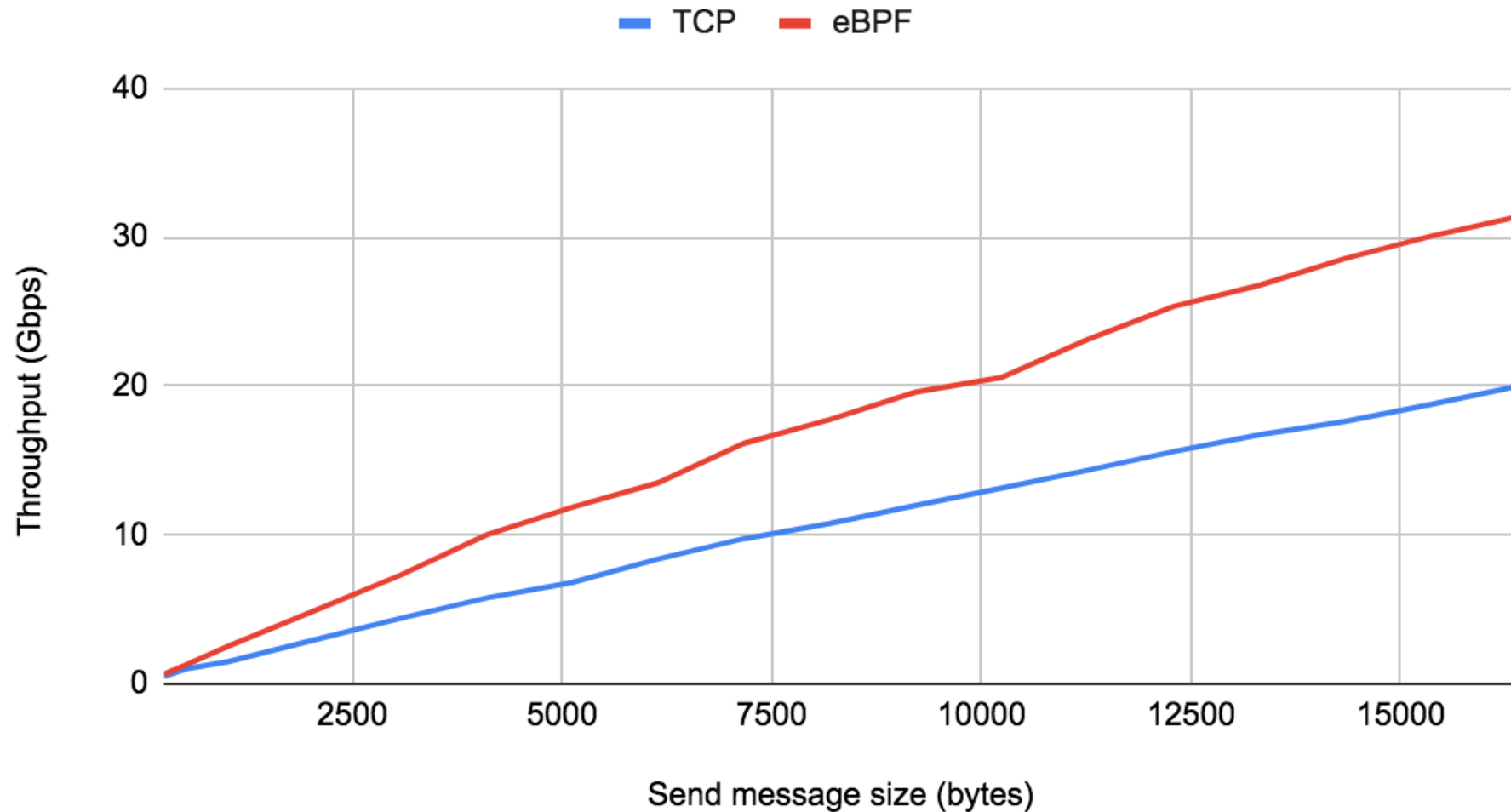
上图显示了eBPF sockhash修改过的TCP/IP与常规TCP/IP堆栈的吞吐量对比。比较有趣得是吞吐量的性能增长对于使用eBPF的发送请求消息大小是线性的。因为当应用程序在发送较大的消息时，几乎没有开销。

这结果与大家的认知不一致：会好奇为什么与常规TCP/IP路径相比吞吐量性能很差。事实证明，罪魁祸首是默认情况下在TCP/IP堆栈中启用了Nagle算法 (<https://tools.ietf.org/html/rfc896>)。引入Nagle算法是为了解决小数据包淹没慢速网络从而导致 congestion 的问题。因为如果它小于TCP MSS的大小，算法只需要一个TCP段是未完成的（未确认），在我们的测试中，这会导致TCP对数据进行批量传输。这种批处理会导致更多的数据传输和分摊开销，并且能够超过 eBPF绕过TCP/IP 的性能收益，它对每个发送调用缓冲区大小有固定的开销（见图7）。

当数据包大小接近TCP MSS时，常规TCP/IP没有合并发送的优势，TCP MSS中可以容纳的数据包更少（测试环境的MSS设置为65483 bytes）并通过以下方式发送到目的地TCP/IP内核堆栈。在这些大数据包发送大小下，eBPF凭借其低开销远远超过使用Nagle算法的TCP/IP堆栈的吞吐。接下来禁用Nagle算法（-D参数）来 netperf 重新测试：

```
netserver -p 1000
netperf -H 127.0.0.1 -p 1000 -l 60 -- -m $msg_size -D
```

TCP with Nagle disabled vs eBPF

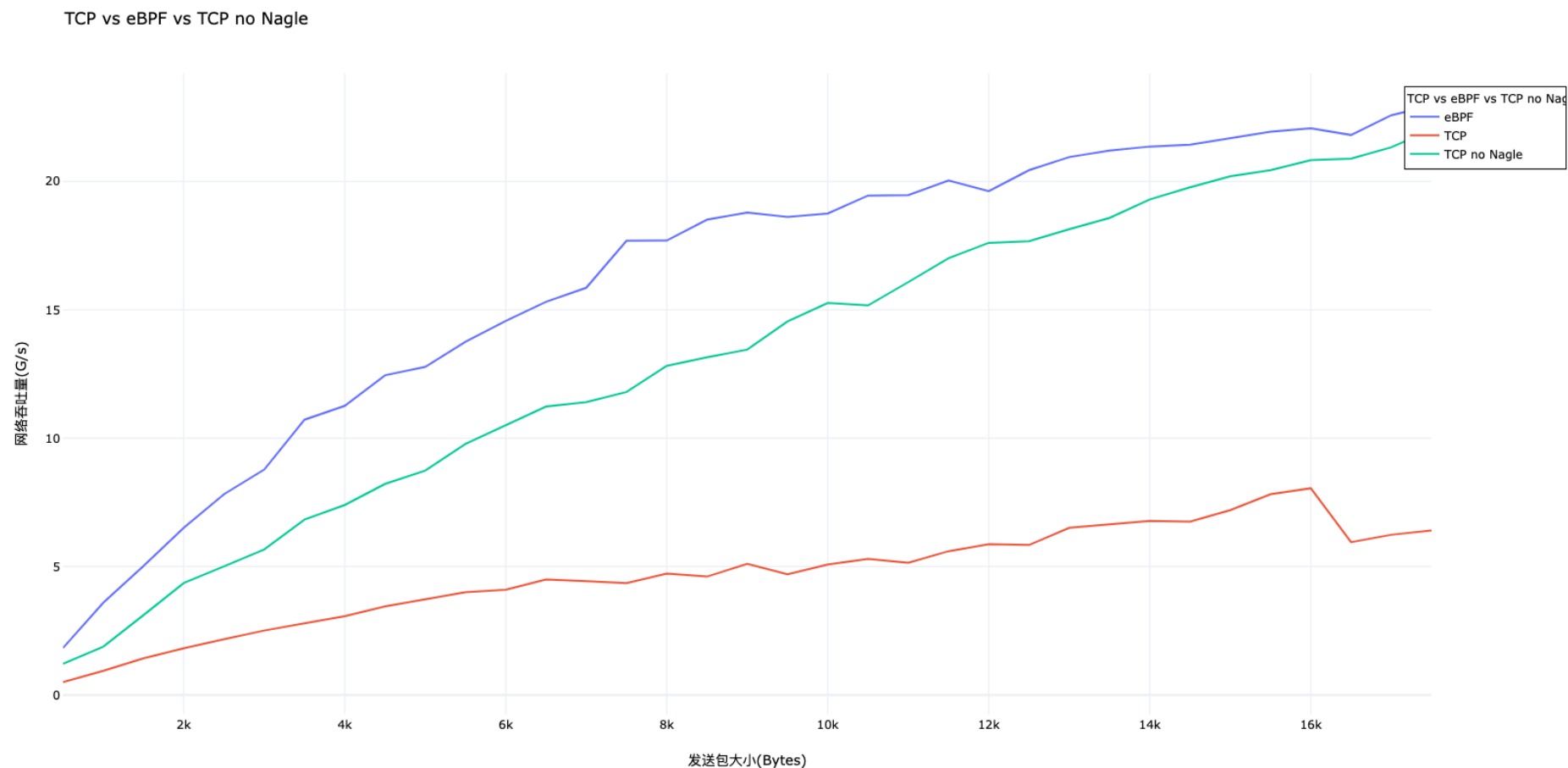


吞吐量：TCP/IP（禁用Nagle算法）与使用 eBPF sockhash 绕过TCP/IP

禁用Nagle算法后，我们看到与eBPF sockhash重定向绕过相比，常规TCP的吞吐量优势完全消失。TCP/IP堆栈和eBPF sockhash映射的性能如预期的那样线性增加——eBPF比常规 TCP/IP具有更大的成长曲线，因为eBPF每次发送调用的固定成本开销。对于较大的发送消息大小和较小的 TCP MSS，这种性能差距更为明显。

译者测试结果

译者在测试时，把三种测试都集中在一张图里了。但测试使用TCP Nagle算法与eBPF绕过的差异并没有很大，而且也没有曲线上的交叉。译者暂时不清楚其中原因，大概是我测试的环境是macbook，恰恰同时我还在使用这台笔记本工作，导致了CPU、网络等不稳定吧。



吞吐量：eBPF VS TCP VS TCP-Disable-Nagle的比较

动态趋势图：



[CONTACT US](#)

Important Update for Chart Studio Users

Chart Studio has shut down

Chart Studio closed on October 31, 2025 at 5pm EDT. We've moved our focus to Plotly Studio and Plotly Cloud, which now serve as the home for AI-assisted visualizations and interactive data apps.

If you need to retrieve your Chart Studio data, please email support@plotly.com with the subject line "Chart Studio data request".

What's next?

本网站利用 Cookie 等技术来启用基本网站功能以及 分析、个性化定制 和 有针对性的广告。要了解更多信息，请查看以下链接：
[隐私政策](#)

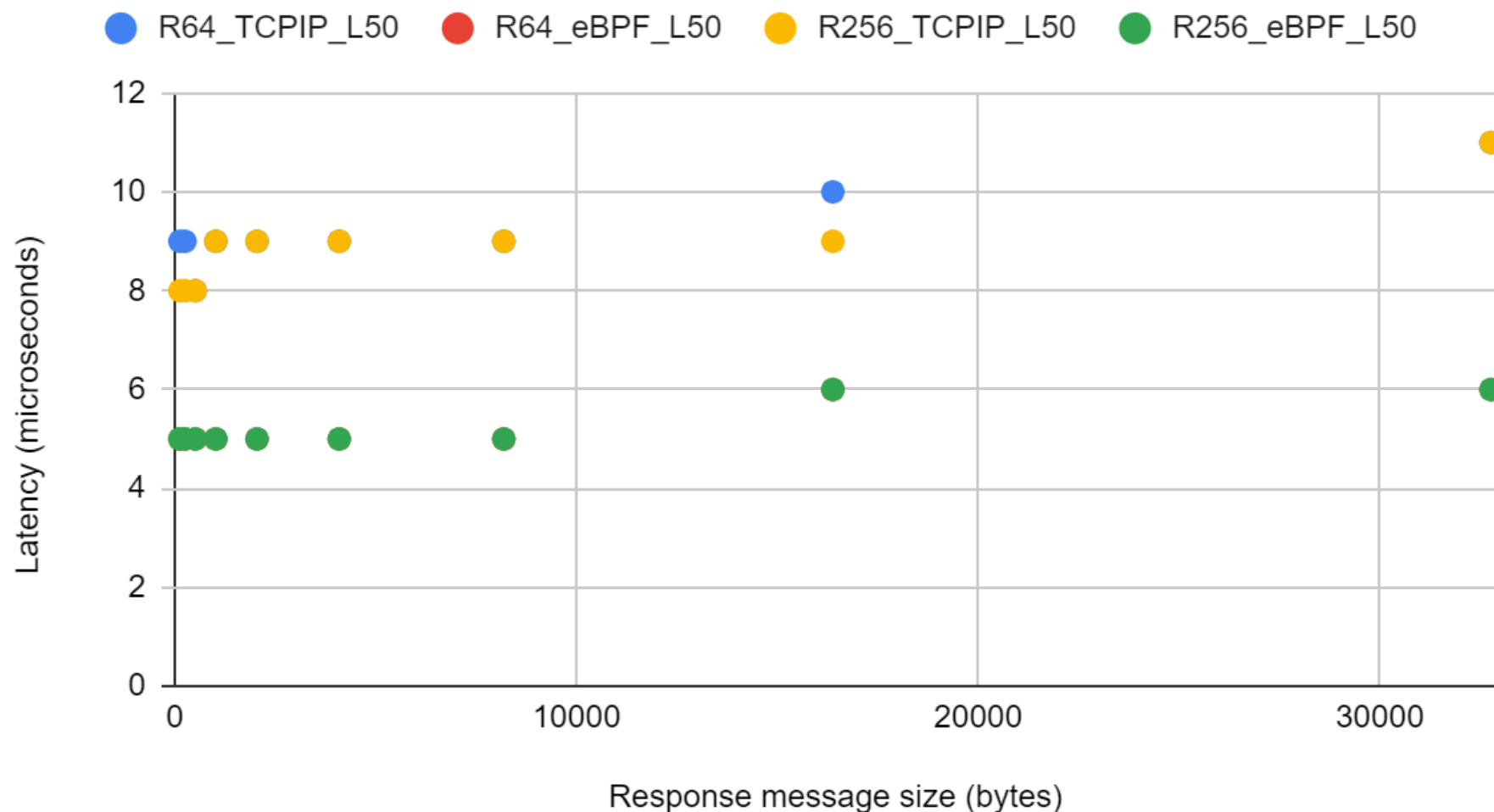
设定偏好



延迟测试结果

使用 `netperf` 重复我们的性能运行来测量延迟。我们测量了延迟的第50个、第90个和第99个百分位数，这会改变发送和接收缓冲区（消息）的大小。测试结果里，使用中位数的延迟来显示趋势图。此外，我们还展示了64字节和256字节的不同请求消息大小的趋势。（请注意：*netperf*一次只有一个事务未完成。）

TCP vs eBPF



延迟测试：使用 eBPF sockhash 绕过 TCP/IP 与 常规 TCP/IP

如图所示，eBPF sockhash 绕过 优于 常规 TCP/IP 堆栈。其性能比常规 TCP/IP 堆栈高出近 50%。考虑到 eBPF 通过将数据包从源 socket 的传输队列重定向到目标 socket 的接收队列，与 TCP/IP 中的那些相比，eBPF 消除了任何协议级别的开销（慢启动、拥塞避免、流量控制等），这些优势带来的结果是显而易见的。此外，请求消息长度对延迟没有影响。我们没有尝试测量接近

TCP MSS 的发送消息大小的延迟。

译者测试结果

译者对这个延迟指标也进行了测试，如下图



eBPF sockhash重定向与常规TCP堆栈的延迟对比

不管是64字节还是256字节的发送消息大小上，都比原生TCP的网络延迟更低。

虽然图表上的结果与原作者测试结果有一定差异，但整体分布比较均衡，也很清楚的体现出eBPF在低延迟上的优势。

动态趋势图



Important Update for Chart Studio Users

Chart Studio has shut down

Chart Studio closed on October 31, 2025 at 5pm EDT. We've moved our focus to Plotly Studio and Plotly Cloud, which now serve as the home for AI-assisted visualizations and interactive data apps.

If you need to retrieve your Chart Studio data, please email support@plotly.com with the subject line "Chart Studio data request".

What's next?

Plotly Studio and Plotly Cloud offer expanded capabilities for building and sharing interactive visualizations and data applications. Both platforms provide modern workflows for teams working with data. Get started with [Plotly Studio](#) or explore [Plotly Cloud](#).

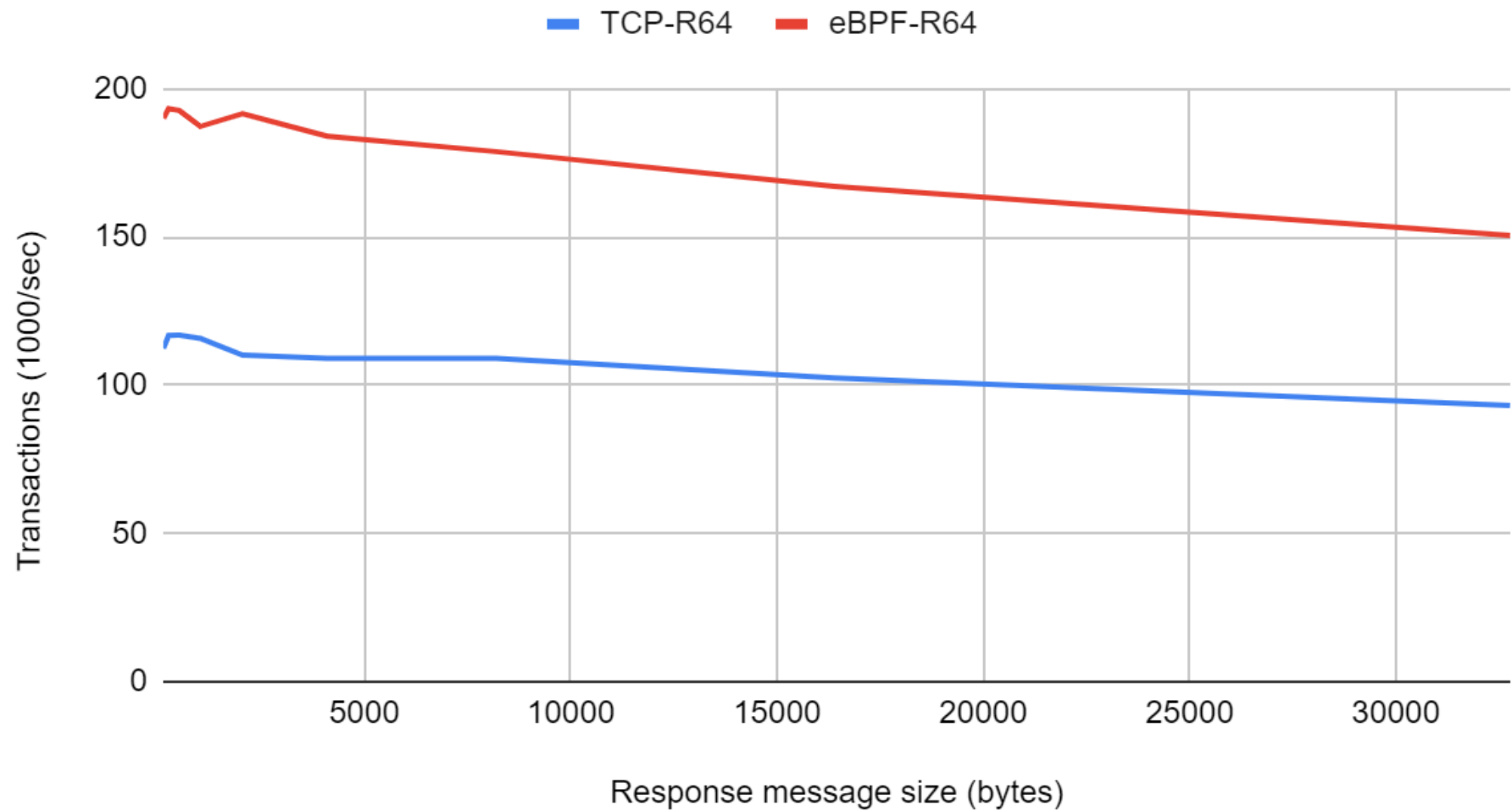


网络事务率

接下来，我们使用`netperf`继续测试事务率：

如果我们简单地将事务率曲线向左翻转，它们的趋势与延迟测量相同。

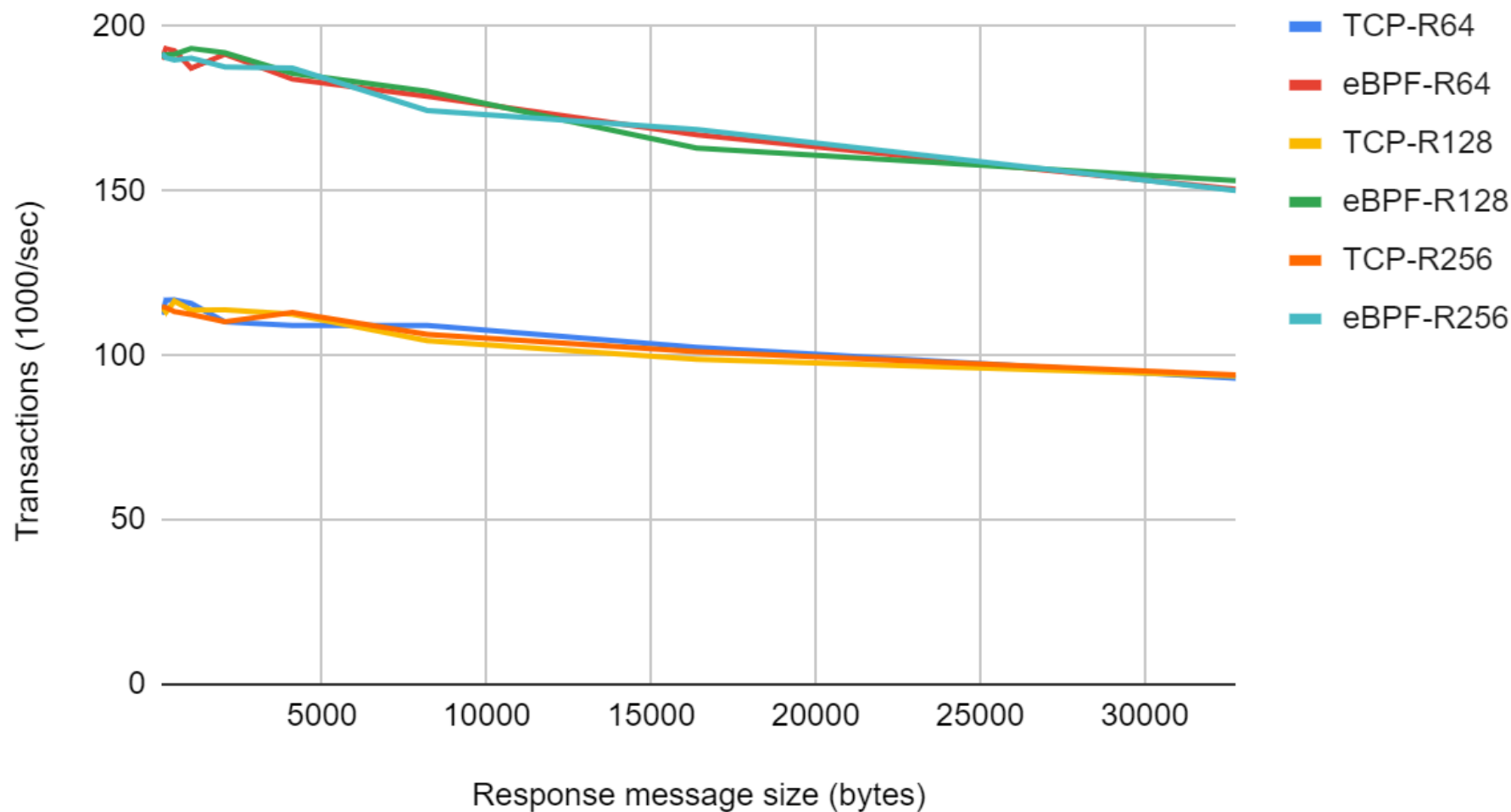
TCP vs eBPF



事务率：使用 eBPF sockhash 重定向与 TCP/IP

如图，针对不同的接收端消息大小绘制了各种发送消息请求大小的事务率。在延迟测量的情况下，我们看到事务率不受常规TCP/IP和eBPF sockhash重定向绕过以及请求响应消息大小的影响。

TCP vs eBPF

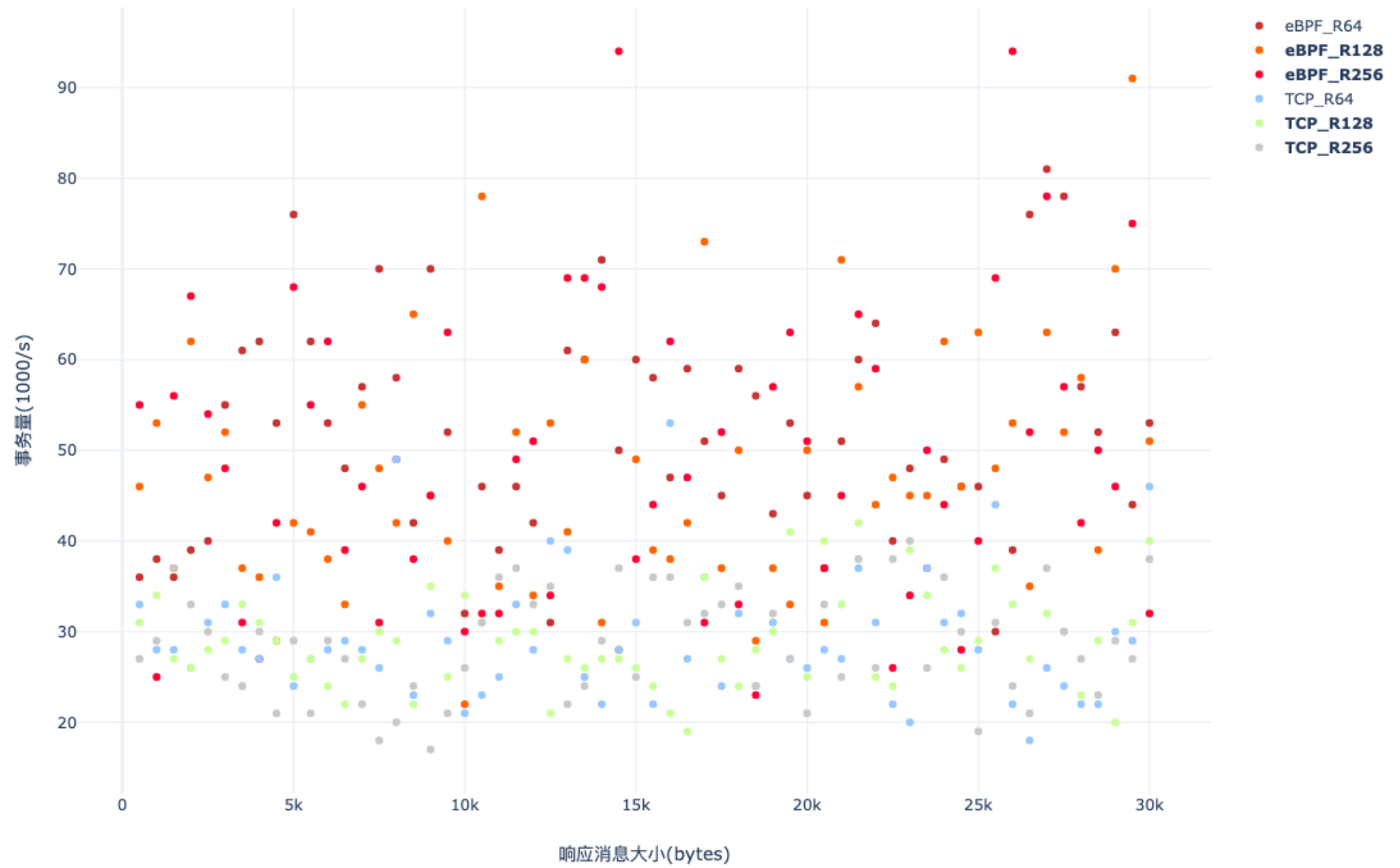


不同请求大小的事务率：TCP/IP vs TCP/IP bypass using eBPF sockhash map

译者测试结果

译者也进行了测试，只是测试环境不太稳定，macbook开的VM虚拟机，在性能测试的时候，同时还在用于办公。测试结果波动比较大，但总体来看，还是能从点的分布看得出eBPF与常规TCP的区别，深色的点是eBPF的结果，都分布在Y轴上面。浅色点是常规TCP的结果，都分布在Y轴下面。如下图：

TCP VS eBPF



动态图表



Important Update for Chart Studio Users

Chart Studio has shut down

Chart Studio closed on October 31, 2025 at 5pm EDT. We've moved our focus to Plotly Studio and Plotly Cloud, which now serve as the home for AI-assisted visualizations and interactive data apps.

If you need to retrieve your Chart Studio data, please email support@plotly.com with the subject line "Chart Studio data request".

What's next?

Plotly Studio and Plotly Cloud offer expanded capabilities for building and sharing interactive visualizations and data applications. Both platforms provide modern workflows for teams working with data. Get started with [Plotly Studio](#) or explore [Plotly Cloud](#).

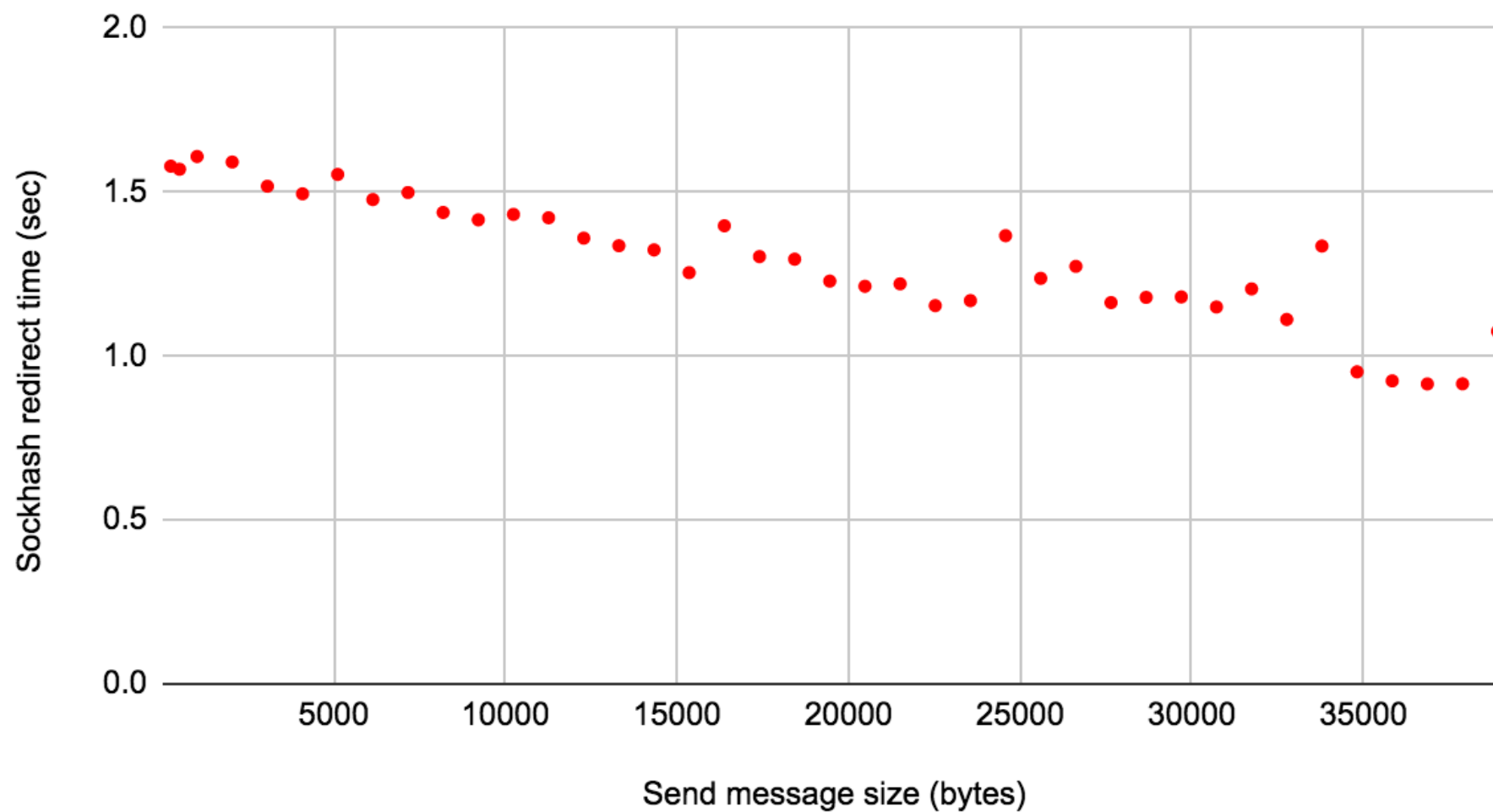


eBPF模式下网络吞吐量测试的内核开销

在下图中，我们绘制了当执行 netperf 吞吐量实验时eBPF sockhash映射绕过在内核中花费的时间。

可以看到对于256字节的发送消息大小，eBPF开销约为1.5秒，并且开销随着发送消息大小的增加而减少。

eBPF overhead



eBPF sockhash map重定向在内核中花费的时间

综述

eBPF确实是一项强大的技术，它允许从用户空间前所未有地访问Linux内核资源。当应用程序对延迟敏感时，使用eBPF绕过TCP/IP堆栈与同一主机上的另一个程序通信是一个最佳的使用场景。尤其是时下最热门的容器编排领域：在Kubernetes集群的同一pod中使用相互依赖的微服务就特别适合这个功能。也就是说，基于eBPF的TCP/IP绕过可以极大地减轻与使用RPC 和 REST API进行大量通信的微服务相关的延迟。

我们还看到，虽然eBPF可以带来强大的性能提升，但盲目使用可能会产生意想不到的后果。具体来说，当我们只关心原始单向吞吐量时，使用默认设置，常规TCP/IP堆栈可以胜过eBPF（Nagle算法合并大量小包的情况）。

译者注：主机安全HIDS的未来趋势

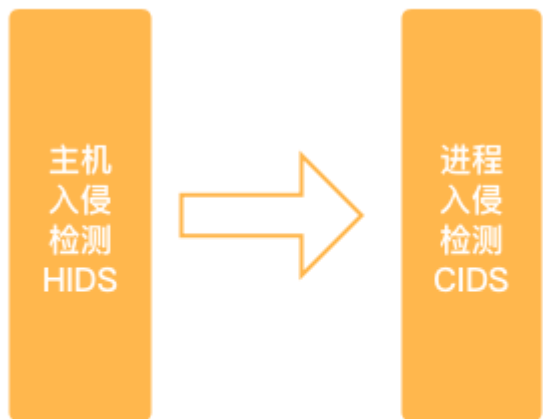
正如本文演示的功能，用eBPF来解决K8S场景中，同一pod的多个docker container之间通讯延迟的问题。那在这种场景下，原来的HIDS产品能否感知到？物理机、虚拟机时代，原来的进程间通讯是IPC、unixdomain socket，而容器化下，是GRPC\HTTP，安全场景上已经有很大变化...

在当下微服务架构日益盛行的时代，必定追求更好性能，更快的速度，更大的负载方向走。而linux kernel的eBPF技术，必定是未来容器化、容器编排领域的热门技术。那么，在信息安全的主机安全领域，HIDS之类产品该何去何从？例如本文中的性能优化，在容器编排场景是特别有价值的使用，甚至已经有了成熟的产品上市。那HIDS还能检测的出来吗？

我们现在使用的单机化、虚拟化的传统思维模式研发安全防护产品，在面对云原生时代容器化的IDC安全问题时，还能跟得上吗？是时候用 CIDS来代替HIDS 了。

注：

- HIDS：Host-based Intrusion Detection System
- CIDS：Container Intrusion Detection System



物理机、虚拟机

容器

附

测试脚本

```
<?php
//benchmark.php
function testTp($req_size) {
    $file = 'result/bench_tp_ebpf_r'.$req_size.'_30s';
    $strCmd = "netperf -t TCP_RR -H 127.0.0.1 -p 1000 -l 30 -- -r $req_size, %d";

    $arrAll = array();
    for ($i=1; $i<=60; $i++)
    {
        $msg_size = $i*500;
        $cmd = sprintf($strCmd,$msg_size);
        echo $file, ": start to exec ",$cmd,"\n";
        $arrExec = array();
        exec($cmd, $arrExec);
        $r = array_pop($arrExec);
        $r = array_pop($arrExec);
        $r1 = preg_split('/\s+/', $r);
        $arrAll[$msg_size] = trim($r1[5]);
        $f = sprintf($file,$msg_size);
    }
    var_dump($arrAll);
    file_put_contents($f,json_encode($arrAll));
}

$arrTest = array(
    64,
    128,
    256);

$strCmdServer = 'netserver -p 1000';
$rs = system($strCmdServer);
var_dump($rs);
foreach ($arrTest as $v) {
```

COPY

```
echo "-----\n";
echo "-----开始测试: $v-----\n";
echo "-----\n";
testTp($v);
}
echo "done\n";
```

结果处理

执行 ./load.sh 脚本, 启用ebpf, 执行 php benchmark.php

合并 result/bench_tp_ebpf_* 中几个文件, 把json的结果转化到csv中

使用 plotly.com 创建图表, 导入csv文件, 开始制表画图

后续

我会开始分析Cilium的eBPF源码 (<https://docs.cilium.io/en/latest/bpf/>), 了解eBPF在K8s产品中实现原理, 更好的研发用eBPF实现行为观测的数据收集, 安全防护的 CIDS(HIDS) 产品



(<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.zh-hans>)CFC4N的博客 由 CFC4N

(<http://www.cnxct.com>) 创作, 采用 署名—非商业性使用—相同方式共享 4.0 (<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.zh-hans>)进行许可。基于<https://www.cnxct.com> (<http://www.cnxct.com>)上的作品创作。转载请注明转自: [译]使用 eBPF (绕过 TCP/IP) 加速云原生应用程序的经验教训 (<https://www.cnxct.com/lessons-using-ebpf-accelerating-cloud-native-zh/>)

暂无相关文章

所谓技术 (<https://www.cnxct.com/category/%e6%89%80%e8%b0%93%e6%8a%80%e6%9c%af/>) cids

(<https://www.cnxct.com/tag/cids/>), container (<https://www.cnxct.com/tag/container/>), docker (<https://www.cnxct.com/tag/docker/>), eBPF (<https://www.cnxct.com/tag/ebpf/>), HIDS (<https://www.cnxct.com/tag/hids/>), k8s (<https://www.cnxct.com/tag/k8s/>), linux kernel

(<https://www.cnxct.com/tag/linux-kernel/>), 主机安全 (<https://www.cnxct.com/tag/%e4%b8%bb%e6%9c%ba%e5%ae%89%e5%85%a8/>), 云原生 (<https://www.cnxct.com/tag/%e4%ba%91%e5%8e%9f%e7%94%9f/>), 容器化 (<https://www.cnxct.com/tag/%e5%ae%b9%e5%99%a8%e5%8c%96/>), 网络安全 (<https://www.cnxct.com/tag/%e7%bd%91%e7%bb%9c%e5%ae%89%e5%85%a8/>). permalink (<https://www.cnxct.com/lessons-using-ebpf-accelerating-cloud-native-zh/>).

[转] 写给工程师：关于证书（certificate）和公钥基础设施（PKI）的一切 (<https://www.cnxct.com/everything-about-pki-zh/>)

[译]使用eBPF per-CPU Cgroup本地存储进行低功耗统计 (<https://www.cnxct.com/ebpf-cgroup-per-cpu-storage-low-overhead-accounting-zh/>)

One thought on “[译]使用eBPF（绕过 TCP/IP）加速云原生应用程序的经验教训”

1. Pingback: [译]使用eBPF per-CPU Cgroup本地存储进行低功耗统计 – CFC4N的博客 (<https://www.cnxct.com/ebpf-cgroup-per-cpu-storage-low-overhead-accounting-zh/>)

Comments are closed.

POPULAR

RECENT

etcd clientv3的lease keepalive租约频繁续期bug 排查小记

(<https://www.cnxct.com/etcd-lease-keepalive-debug-note/>)

7 月 18, 2018

eCaptureQ：给1.5万星标的开源项目
eCapture装个酷炫的驾驶舱！

(<https://www.cnxct.com/ecaptureq/>)

11 月 2, 2025

eCapture开源项目三周年， v1.0.0 稳定版
发布 (<https://www.cnxct.com/ecapture-v1-0-0/>)

8 月 22, 2025

eCapture旁观者：Android HTTPS明文抓包，无需CA证书

(<https://www.cnxct.com/ecapture-for-android/>)

9 月 13, 2022

eCapture v0.8.0：CO-RE、非CO-RE二合一，交叉编译amd64、arm64双版本
(<https://www.cnxct.com/ecapture-v0-8-0-all-in-one/>)

5 月 6, 2024

安全

黑客Depy的博客 (<https://rce.ink>)

程序

codedump (<https://www.codedump.info>)

运维

王一的Blog (<http://www.mudone.com/>)

其他操作

登录 (<https://www.cnxct.com/wp-login.php>)

条目 feed (<https://www.cnxct.com/feed/>)

评论 feed

(<https://www.cnxct.com/comments/feed/>)

WordPress.org (<https://cn.wordpress.org/>)
